Supplementary Material: STC: Inference-Time Semantic Target Control for Knowledge Graph Completion Rankings

Efstratios Skaperdas[0009-0004-6423-0240] and Nick Bassiliades[0000-0001-6035-1038]

School of Informatics, Aristotle University of Thessaloniki, Thessaloniki, Greece
eskapera@csd.auth.gr nbassili@csd.auth.gr

Annex Overview

The main paper is self-contained. The annexes provide the theoretical, experimental, and reproducibility detail needed to inspect and regenerate the reported results. Annex A describes dataset construction and semantic signals; Annex B gives extended proofs and controller details; Annex C reports frozen-model and configuration details; Annex D provides additional results and artifact information; Annex E organizes the extended empirical diagnostics; and Annex F reports the exact semantic-evidence coverage inventory. Table 1 separates the contents of the paper, this supplementary PDF, the public code artifact, the large-artifact archive, and restricted source data.

A Dataset Construction and Semantic Signals

We evaluate on EurostatKG, DrugBank, and DBpedia. EurostatKG is used through a processed link-prediction artifact bundle derived from the released EurostatKG resources. DrugBank is constructed from the DrugBank XML dump by extracting entity-valued links between drugs, proteins, and pathways. DBpedia is processed from the DBpedia 2016-10 ontology, English instance types, and English mapping-based object triples; models are trained and selected on the full processed dataset, while semantic-control evaluations are run on a fixed stratified subset of test query slots for tractability. In all cases, candidate generation and all downstream control methods operate on frozen scores and identical candidate windows, ensuring controlled comparisons across datasets. Table 2 summarizes the final train/validation/test statistics for the processed benchmarks.

Dataset sources and versions. EurostatKG is derived from released EurostatKG resources and represented as a processed link-prediction artifact bundle. DrugBank is derived from the DrugBank full database XML (version 5.1, accessed/exported on 2025-01-02) [3]. Due to licensing restrictions, the raw XML dump is not redistributed; users with authorized access can regenerate the derived dataset using the supplied extraction scripts and configuration files. DBpedia is

Table 1. Distribution and availability of the supporting material.

Component	Contents	Location / availability
Main paper	Core formulation, guarantees, protocol, principal comparisons, uncertainty, runtime, and limitations	Proceedings PDF
Supplementary PDF	Full proofs, dataset and semantic-signal documentation, frozen-model details, extended analyses, diagnostics, and example certificates	This document
Public code artifact	Reference implementation, scripts, configurations, environments, lightweight metadata, curated results, and provenance checks	GitHub repository
Large-artifact archive	Redistributable processed datasets, frozen checkpoints, materialized candidate windows, full query-level and verified experiment outputs, and execution records	External Drive folder
Restricted source data	DrugBank XML and any other source files whose licenses prohibit redistribution	Not redistributed; obtained from the original provider

Table 2. Dataset statistics used in the experiments.

Dataset	**Entities**	**Relations**	**Train**	**Valid**	**Test**
EurostatKG	44,948	20	51,709	6,461	6,461
DrugBank	22,466	6	2,343,802	292,976	292,976
DBpedia	5,496,987	660	14,662,925	1,832,876	1,832,876

reconstructed from the documented DBpedia 2016-10 ontology, English instance-type, and English mapping-based object-triple files [1] (`dbpedia_2016-10.owl`, `instance_types_en.ttl.bz2`, and `mappingbased_objects_en.ttl.bz2`) using the supplied preprocessing scripts. Redistributable source and derived artifacts are made available through the `large-artifact archive`.

EurostatKG provenance. EurostatKG is based on a previously released ontology-based knowledge graph for Eurostat and selected OECD resources [2]. The source graph models statistical articles, datasets, classifications, references, themes, glossary terms, and related metadata, and was reported to contain more than 820K triples, including explicit and inferred knowledge. In this work, we use a processed link-prediction version of the graph, where entity-valued RDF statements are converted into triples suitable for KGC training and evaluation. The processed version provides the entity and relation identifiers, train/validation/test splits, and ontology-derived side information needed for semantic admissibility checks. Table 3 summarizes the source and processed benchmark.

EurostatKG preprocessing. EurostatKG is processed from RDF/OWL resources into a unified link-prediction artifact bundle. The preprocessing script reads graph files and, when available, ontology/schema files; extracts entity-

Table 3. EurostatKG source and processed benchmark summary.

Quantity	Value
Original source	Previously released Eurostat Knowledge Graph [2]
Domain	Statistical articles, datasets, classifications, references, themes
Original KG scale	More than 820K triples reported by the source paper
External resources	Eurostat and selected OECD resources
Processed task format	Link-prediction triples
Entities	44,948
Relations	20
Train/valid/test triples	51,709/6,461/6,461
Split seed	42

valued KG triples; separates schema-level statements such as `rdf:type`, `rdfs:subClassOf`, `rdfs:domain`, `rdfs:range`, and `owl:disjointWith`; and writes the resulting train/validation/test splits, entity and relation mappings, entity types, relation constraints, disjoint pairs, file-level parsing statistics, and a preflight summary. Type information is materialized using the subclass closure before semantic admissibility checks. Triples are split by relation using an 0.8/0.1/0.1 train/validation/test ratio with seed 42.

DrugBank construction. DrugBank is constructed from the DrugBank XML structure by extracting entity-valued links between drugs, proteins, and pathways. The extracted relation signatures are shown below, and Table 4 summarizes the resulting XML-derived KG construction.

$$
\begin{aligned}
&\texttt{drug\_interacts\_with}: && \textsc{Drug} \to \textsc{Drug},\\
&\texttt{drug\_has\_target}: && \textsc{Drug} \to \textsc{Protein},\\
&\texttt{drug\_has\_enzyme}: && \textsc{Drug} \to \textsc{Protein},\\
&\texttt{drug\_has\_carrier}: && \textsc{Drug} \to \textsc{Protein},\\
&\texttt{drug\_has\_transporter}: && \textsc{Drug} \to \textsc{Protein},\\
&\texttt{drug\_in\_pathway}: && \textsc{Drug} \to \textsc{Pathway}.
\end{aligned}
$$

Entity types are induced from the XML-derived entity prefixes, and relation signatures define the domain–range constraints used for semantic checking. Disjointness is defined over the three unordered type pairs among Drug, Protein, and Pathway. Duplicate triples are removed, and triples are split into train, validation, and test sets using a fixed random seed, stratified by relation. Due to DrugBank licensing restrictions, we do not redistribute the original XML dump; instead, we provide the extraction scripts and configuration needed to reconstruct the derived KG from an authorized DrugBank download.

DBpedia preprocessing. DBpedia is processed from the DBpedia 2016-10 ontology, English instance-type file, and English mapping-based object triples. We retain resource-to-resource object triples whose head and tail satisfy the configured DBpedia resource prefix filter, and deduplicate the resulting triples, using them as KGC triples. The ontology file provides subclass, domain, range, `owl:disjointWith`, and `owl:disjointUnionOf` information; instance types are

Table 4. DrugBank XML-derived KG construction summary.

Quantity	Value
Input source	DrugBank full database XML
Entity types	Drug, Protein, Pathway
Train/valid/test ratio	0.8/0.1/0.1
Split seed	42
Extracted relation types	6
Disjoint type pairs	3
Unique triples	2,929,754
Entities	22,466
Train triples	2,343,802
Valid triples	292,976
Test triples	292,976

Table 5. DBpedia preprocessing summary.

Quantity	Value
Original triples considered	18,328,677
Retained triples after prefix filtering and deduplication	18,328,677
Retained entities	5,496,987
Retained relations	660
Schema constraints retained	domain/range/disjointness

materialized through the transitive subclass closure. The resulting artifacts include train/validation/test splits, entity and relation mappings, materialized entity types, relation domain/range constraints, disjoint pairs, relation split statistics, source-file statistics, and a preflight summary. Splits are created relation-wise using validation and test fractions of 0.1 each with seed 42. Table 5 summarizes the resulting DBpedia preprocessing statistics.

Preprocessing statistics. For transparency and reproducibility, each preprocessing pipeline produces a `preflight.json` summary containing dataset statistics, schema coverage, typing coverage, relation distributions, and runtime diagnostics. All statistics reported in Tables 2, 3, 4, and 5 are derived from these summaries and the corresponding split/statistics files.

Processed artifacts. Table 6 summarizes the common files produced by the preprocessing pipelines. The `all_triples.tsv` file contains the complete processed KG before splitting, while `train.tsv`, `valid.tsv`, and `test.tsv` define the link-prediction splits. The mapping files `entity2id.json` and `relation2id.json` assign integer identifiers used by the KGC models. The semantic sidecars `entity_types.json`, `relation_constraints.json`, and `disjoint_pairs.json` provide the type assertions, domain/range constraints, and disjointness information used for admissibility checks. The `relation_split_stats.json` file records per-relation split counts. Dataset-specific diagnostic files record preprocessing

Table 6. Processed artifact bundle produced for each dataset.

Artifact	EurostatKG	DrugBank	DBpedia
`all_triples.tsv`	Yes	Yes	Yes
`train.tsv / valid.tsv / test.tsv`	Yes	Yes	Yes
`entity2id.json / relation2id.json`	Yes	Yes	Yes
`entity_types.json`	Yes	Yes	Yes
`relation_constraints.json`	Yes	Yes	Yes
`disjoint_pairs.json`	Yes	Yes	Yes
`relation_split_stats.json`	Yes	Yes	Yes
`file_stats.json`	Yes	–	Yes
`parse_stats.json`	–	Yes	–
`preflight.json`	Yes	Yes	Yes

details: `file_stats.json` for RDF/OWL-based datasets and `parse_stats.json` for the XML-derived DrugBank pipeline. Finally, `preflight.json` provides a compact run summary with counts, configuration, schema coverage, and runtime diagnostics. These sidecar files are loaded by the experiment scripts to ensure that semantic checks, feasibility diagnostics, and all control methods use the same processed ontology-derived information.

DBpedia evaluation subset. The embedding model is trained, validated, and selected on the full processed dataset. To make semantic-control evaluation tractable under large candidate windows, we evaluate on a fixed stratified subset of test query slots rather than on all possible test queries. The subset is not used for training or model selection. It is derived from the test split by creating both head and tail prediction slots for each test triple and then sampling up to a fixed cap per relation and prediction mode. This subset is used exclusively for evaluation and does not affect training or model selection. The subset construction preserves the original relation distribution while bounding evaluation cost. Table 7 reports the subset construction details. All methods are evaluated on exactly the same sampled query slots and candidate windows, so the subset affects only evaluation cost, not comparative fairness.

Candidate windows. All methods operate on the same fixed materialized candidate windows $\mathcal{W} = \text{Top-}M$ extracted from frozen model scores; no method re-queries or expands a window during control. Unless otherwise stated, $k = 10$. For DBpedia, the candidate windows are computed for the fixed evaluation query subset described above. Table 8 gives the candidate-window sizes used in the experiments.

A.1 Ontology Constraints and Semantic Energies

For a query slot, e.g., $(h, r, ?)$ with candidate tail t or $(?, r, t)$ with candidate head h, admissibility is determined by ontology-derived checks. A candidate is admissible when it satisfies the applicable schema constraints for relation r. Type information is obtained from the processed knowledge graph, including explicit type assertions and materialized subclass closure when available. For

Table 7. DBpedia evaluation-subset construction.

Quantity	**Value**
Training data	Full processed DBpedia train split
Validation data	Full processed DBpedia validation split
Model selection	Full validation split
Evaluation source split	Test
Prediction modes	Head and tail
Sampling strategy	Stratified by relation and mode
Cap per relation/mode	50
Sampling seed	42
Test triples before sampling	1,832,876
Query slots before sampling	3,665,752
Selected query slots	47,246
Relations represented	645

Table 8. Candidate-window sizes.

Dataset	k	M
EurostatKG	10	20,000
DrugBank	10	5,000
DBpedia	10	5,000

a fixed query (h, r), the semantic energy is defined per candidate $t \in \mathcal{W}$ as $e(h, r, t)$, which we write as $e(t)$ when the query is clear from context. The ontology-derived checks are computed from the processed artifact bundle, using the entity types (`entity_types.json`), relation domain/range constraints (`relation_constraints.json`), and disjointness pairs (`disjoint_pairs.json`).

Candidate-level statuses are *admissible*, *violating*, and *unknown*. Returned-list semantic validity is the list-level property assessed from these statuses, for example through the admissible count, violation rate, or quota satisfaction. A violating status means failure of an operational candidate-level admissibility check; it is not an OWL open-world inconsistency claim or a factual-error judgment. Unknown status records insufficient evidence and is not relabeled as violation.

The main checks are:

- domain compatibility for the head entity;
- range compatibility for the tail entity;
- disjointness violations between inferred or asserted types;
- unknown status when required type information is missing for constraint evaluation.

Unless otherwise stated, the main reported experiments use domain–range constraints; disjointness is evaluated separately in the constraint-family ablations.

Binary energy. The main experiments use a binary semantic energy:

$$e(t) = \begin{cases} 0 & \text{if } t \text{ is admissible,} \\ 1 & \text{if } t \text{ is violating or unknown.} \end{cases}$$

Table 9. Semantic-energy assignment.

Candidate status	Main energy	Unknown-aware energy
Admissible	0	0
Unknown	1 (binary)	β
Violating	1	≥ 1

All explicit violations receive the same positive penalty, set to 1 unless otherwise stated. The exact penalty magnitude for violating candidates does not affect the feasibility or minimal-intervention guarantees, as long as it is strictly positive.

In the fully checkable setting, violating candidates receive a positive penalty and admissible candidates receive zero penalty. This is the setting used for the main finite-window guarantees and collapse analysis.

Unknown-aware energy. For incomplete schema information, we evaluate a graded energy:

$$e(t) = \begin{cases} 0 & \text{admissible,} \\ \beta & \text{unknown,} \\ \geq 1 & \text{violating,} \end{cases} \qquad \beta \in (0, 1].$$

The unknown-aware controller OptK-UA softly penalizes uncertainty while still penalizing explicit violations more strongly. Unknown status arises when either the head or tail entity lacks the required type assertions (after subclass materialization) to evaluate the applicable domain, range, or disjointness constraints. These energies instantiate the semantic signal used in the controllability formulation, directly inducing the score perturbations analyzed in the main paper. The binary setting corresponds to the theoretical analysis in Section 4.2, while the graded variant generalizes the control signal to partially observed semantic information. Note that the semantic energy depends on the query context (h, r) through the applicable relation-specific constraints and we write $e(t)$ for brevity.

Table 9 summarizes the semantic-energy assignment used in the experiments. In the binary setting, unknown candidates retain status unknown but are assigned the same positive energy as checkable violations unless otherwise specified.

B Extended Proofs and Controller Details

B.1 Full Proofs of the Controllability Results

This section provides full proofs and useful special cases of the controllability result stated as Proposition 1 in the main paper. The first two arguments establish its finite-window feasibility and minimum-pressure parts. All results are stated for a fixed query and a finite candidate window $\mathcal{W} = \text{Top-}M$. We write $s(t)$ for the frozen base score, $e(t)$ for the semantic energy, and

$$s_\lambda(t) = s(t) - \lambda e(t), \qquad \lambda \geq 0.$$

Under the zero-versus-positive energy structure assumed by the guarantees, let $\mathcal{A} = \{t \in \mathcal{W} : e(t) = 0\}$ denote the admissible candidates and $\mathcal{P} = \{t \in \mathcal{W} :$

$e(t) > 0\}$ the positive-energy candidates. All top-k operators are interpreted with a fixed deterministic tie-breaking rule. Let τ_q denote the score of the q-th highest-scoring admissible candidate in $\mathcal{A}$ under the fixed deterministic tie-breaking rule.

Finite-window feasibility (Proposition 1) We show that a quota $q \leq k$ is achievable if and only if $|\mathcal{A}| \geq q$.

First, suppose that $|\mathcal{A}| < q$. Since all returned candidates must belong to the finite window $\mathcal{W}$, no ranking or score transformation over $\mathcal{W}$ can place q admissible candidates in the returned top-k. Thus the target is infeasible.

Conversely, suppose that $|\mathcal{A}| \geq q$. Let τ_q be the score of the q-th highest-scoring admissible candidate in $\mathcal{A}$ under the fixed deterministic tie-breaking rule. Since every positive-energy candidate $v \in \mathcal{P}$ satisfies $e(v) > 0$, we have

$$s_\lambda(v) = s(v) - \lambda e(v) \to -\infty \quad \text{as } \lambda \to \infty.$$

Since $\mathcal{W}$ is finite and $e(v) > 0$ for all $v \in \mathcal{P}$, there exists a finite $\lambda \geq 0$ such that every positive-energy candidate has controlled score below τ_q, while admissible scores remain unchanged because $e(a) = 0$ for all $a \in \mathcal{A}$. Hence the q admissible candidates defining τ_q appear above all positive-energy candidates, and the returned top-k contains at least q admissible items. Therefore the quota is achievable exactly when $|\mathcal{A}| \geq q$. □

Minimum-pressure guarantee (Proposition 1) Assume $|\mathcal{A}| \geq q$ and let τ_q be the score of the q-th highest-scoring admissible candidate under the fixed deterministic ordering. Set

$$b = k - q,$$

the number of candidates not classified as admissible that the returned top-k may contain while still satisfying the quota. For each positive-energy candidate $v \in \mathcal{P}$ that initially scores above τ_q, define its positive threshold crossing

$$\gamma_q(v) = \frac{s(v) - \tau_q}{e(v)} > 0.$$

Let Γ_q be the multiset of these positive crossings, sorted in non-increasing order,

$$\Gamma_{q,(1)} \geq \Gamma_{q,(2)} \geq \cdots \geq \Gamma_{q,(m)}, \qquad m = |\Gamma_q|.$$

The controller uses

$$\lambda_q^\star = \begin{cases} 0, & m \leq b, \\ \Gamma_{q,(b+1)}, & m > b. \end{cases}$$

The controlled ranking is ordered first by decreasing controlled score $s_\lambda(t)$, then by lower semantic energy, then by decreasing frozen score, and finally by

the original candidate index. Under this ordering, a positive-energy candidate v is strictly above the q-th admissible threshold after intervention exactly when

$$s(v) - \lambda e(v) > \tau_q \quad \Longleftrightarrow \quad \gamma_q(v) > \lambda.$$

A candidate with $\gamma_q(v) = \lambda$ ties at τ_q and is placed after the admissible threshold candidate because lower energy is preferred.

If $m \leq b$, at most b positive-energy candidates are initially above τ_q, so the base ranking already contains at least q admissible candidates in its top-k and $\lambda_q^\star = 0$ is sufficient. Otherwise, at $\lambda = \Gamma_{q,(b+1)}$, at most the first b crossings are strictly larger than λ. Hence at most b positive-energy candidates remain strictly above τ_q. Together with the $q-1$ admissible candidates ordered ahead of the q-th admissible candidate, this places that candidate within the first

$$(q-1) + b + 1 = k$$

positions, so the returned top-k contains at least q admissible items.

To prove minimality, suppose $m > b$ and let $\lambda < \Gamma_{q,(b+1)}$. Then at least $b+1$ positive crossings satisfy $\gamma_q(v) > \lambda$, so at least $b+1$ positive-energy candidates remain strictly above τ_q. Since $q-1$ admissible candidates also precede the q-th admissible candidate, at least

$$(q-1) + (b+1) = k$$

candidates precede it. The returned top-k therefore contains fewer than q admissible items. No smaller non-negative intervention can satisfy the quota.

Thus $\lambda_q^\star$ is the minimum intervention within the scalar controlled-score family $s(t) - \lambda e(t)$ under the stated deterministic tie-breaking rule. This is not a minimum edit-distance claim over arbitrary permutations. □

Exact-quota specialization ($q = k$) Set $q = k$. If $|\mathcal{A}| \geq k$, the finite-window feasibility argument above shows that the exact quota is feasible within $\mathcal{W}$. The minimum-pressure argument then shows that $\lambda_k^\star$ is sufficient to ensure that no positive-energy candidate scores above the k-th admissible threshold τ_k. Hence the returned top-k contains at least k admissible candidates. Since the returned list has exactly k items, all returned items are admissible. □

Equal-energy collapse to admissibility-first Assume that every candidate not classified as admissible has the same positive energy:

$$e(t) = \begin{cases} 0 & \text{if } t \text{ is admissible,} \\ \eta > 0 & \text{otherwise.} \end{cases}$$

This includes the canonical binary setting: unknown candidates retain status unknown but share the same positive energy as violating candidates. For every admissible candidate $a \in \mathcal{A}$,

$$s_\lambda(a) = s(a),$$

whereas for every positive-energy candidate $v \in \mathcal{P}$,

$$s_\lambda(v) = s(v) - \lambda\eta.$$

Thus the internal score order among admissible candidates is unchanged, and the internal score order among positive-energy candidates is also unchanged.

Because $\mathcal{W}$ is finite and $|\mathcal{A}| \geq k$, let a_k denote the k-th highest-scoring admissible candidate. Since $\eta > 0$, there exists a finite λ large enough such that, for every positive-energy candidate $v \in \mathcal{P}$,

$$s(v) - \lambda\eta < s(a_k).$$

For such λ, all top-k positions are occupied by admissible candidates, and their internal order is exactly their original score order. Therefore the returned top-k consists of the top-k admissible candidates in base-score order, which is precisely admissibility-first ranking. Hence, in the exact feasible case, OptK collapses to AdmissibleFirst. □

Additional structural property: quota monotonicity

Lemma 1 (Quota monotonicity). *If $q_1 \leq q_2$ and both quotas are feasible, then*

$$\lambda^\star_{q_1} \leq \lambda^\star_{q_2}.$$

Proof. Any value of λ that returns at least q_2 admissible candidates also returns at least q_1 admissible candidates because $q_1 \leq q_2$. Therefore the set of scalar interventions satisfying quota q_2 is a subset of the set satisfying quota q_1. Taking the minimum non-negative intervention over the two nested feasible sets gives

$$\lambda^\star_{q_1} \leq \lambda^\star_{q_2}.$$

Thus stricter quotas require weakly larger intervention. □

Additional structural property: finite breakpoints

Lemma 2 (Piecewise-constant rankings). *As a function of λ, the ranking induced by s_λ is piecewise constant over finitely many intervals.*

Proof. For any two candidates $u, v \in \mathcal{W}$, their relative order can change only when

$$s(u) - \lambda e(u) = s(v) - \lambda e(v).$$

If $e(u) \neq e(v)$, this equality has the unique solution

$$\lambda = \frac{s(u) - s(v)}{e(u) - e(v)}.$$

If $e(u) = e(v)$, their score difference is independent of λ, so their relative order never changes except for ties already resolved by the fixed tie-breaking rule. Since $\mathcal{W}$ is finite, there are only finitely many candidate pairs and therefore finitely many possible breakpoints. Between consecutive breakpoints, no pairwise order changes, so the induced ranking is constant. □

Algorithm 1 Extended STC controller for $(h, r, ?)$

Require: Frozen scorer f_Θ, window size M, output size k, quota $q \le k$, semantic energy e
Ensure: Controlled top-k list $\widehat{Y}_k$ and certificate $\mathcal{C}$
$\mathcal{W} \leftarrow$ Top-M under $f_\Theta(h, r, \cdot)$
Compute $s(t) = f_\Theta(h, r, t)$ and $e(t)$ for all $t \in \mathcal{W}$
$\mathcal{A} \leftarrow \{t \in \mathcal{W} : e(t) = 0\}$
if $|\mathcal{A}| < q$ **then**
    $\mathcal{C} \leftarrow (q, |\mathcal{A}|, \perp, \perp, \textsc{Infeasible})$
    return Top-$k(s)$ with deterministic tie-breaking and $\mathcal{C}$
end if
Sort $\mathcal{A}$ by decreasing $s(t)$, then original candidate index
Let t_q be the q-th element in $\mathcal{A}$ and set $\tau_q \leftarrow s(t_q)$
$b \leftarrow k - q$
$\Gamma_q \leftarrow \mathrm{sort}_\downarrow \left\{ \frac{s(t)-\tau_q}{e(t)} : e(t) > 0,\ s(t) > \tau_q \right\}$
if $|\Gamma_q| \le b$ **then**
    $\lambda_q^\star \leftarrow 0$
else
    $\lambda_q^\star \leftarrow \Gamma_{q,(b+1)}$
end if
Define $s_\lambda(t) = s(t) - \lambda_q^\star e(t)$
Sort $\mathcal{W}$ by decreasing $s_\lambda(t)$, lower $e(t)$, decreasing $s(t)$, then original candidate index
$\widehat{Y}_k \leftarrow$ first k elements of the sorted list
$\mathcal{C} \leftarrow (q, |\mathcal{A}|, \tau_q, \lambda_q^\star, \textsc{Feasible})$
return $\widehat{Y}_k$ and $\mathcal{C}$

B.2 Extended Inference-Time Controller

Algorithm 1 provides a fully explicit specification of the inference-time controller, including feasibility handling, deterministic tie-breaking, numerical conventions, and the exact correspondence to the theoretical results.

Setting. For a fixed query $(h, r, ?)$, let $\mathcal{W} =$ Top-M denote the finite candidate window produced by the frozen scorer $f_\Theta(h, r, \cdot)$. Each candidate $t \in \mathcal{W}$ is associated with a base score $s(t)$ and a semantic energy $e(t) \ge 0$. The admissible set is $\mathcal{A} = \{t \in \mathcal{W} : e(t) = 0\}$.

Unless otherwise specified, the controller operates in the *binary energy setting*, where unknown candidates retain status unknown but receive the same positive energy as violating candidates. The controller generalizes directly to graded energies.

Goal. Given a quota $q \le k$, the controller returns a top-k list that contains at least q admissible candidates whenever this is feasible within $\mathcal{W}$.

Explanation. The controller first extracts the fixed candidate window $\mathcal{W} =$ Top-M and computes frozen scores and semantic energies. If $|\mathcal{A}| < q$, the target is infeasible within the materialized window; the deployment protocol therefore returns the frozen base top-k together with an infeasibility certificate.

For a feasible target, the controller sets $b = k - q$ and computes only the positive threshold crossings of positive-energy candidates that initially score

above the q-th admissible threshold. If no more than b such crossings exist, the frozen top-k already satisfies the quota and no intervention is applied. Otherwise, the $(b+1)$-st largest crossing is the minimum scalar pressure that leaves at most b positive-energy candidates strictly above τ_q. The controlled ranking then uses the deterministic ordering specified in the algorithm.

This procedure implements the partial-quota controller analyzed in the main paper. It operates only on the already materialized finite top-M window: there is no retraining, no re-querying, and no candidate-window expansion at the control stage.

Certificate. The certificate is

$$\mathcal{C} = (q, |\mathcal{A}|, \tau_q, \lambda_q^\star, \text{status}),$$

where q is the requested quota, $|\mathcal{A}|$ is the admissible count in $\mathcal{W}$, τ_q is the admissible threshold when feasible, $\lambda_q^\star$ is the minimum intervention in the scalar family $s(t) - \lambda e(t)$, and status $\in \{\textsc{Feasible}, \textsc{Infeasible}\}$.

Determinism and numerical conventions. The crossing multiset contains only candidates with $e(t) > 0$ and $s(t) > \tau_q$, so division by zero and non-positive crossings are excluded. Controlled rankings are ordered by controlled score, lower energy, frozen score, and original candidate index. This ordering makes ties at τ_q quota-safe and fully reproducible.

Complexity. After scores and energies are available, constructing $\mathcal{A}$ and the crossing multiset requires a linear scan over $\mathcal{W}$. The required crossing can be obtained by selection or sorting, and producing the final controlled ranking requires sorting or top-k selection. The controller is therefore independent of model training and has at most linearithmic overhead in the window size.

C Frozen Models and Experimental Configuration

C.1 Frozen Models and Candidate Extraction

Setting and notation. For each dataset, we consider standard link-prediction queries of the form $(h, r, ?)$ and $(?, r, t)$ derived from the test split. Let Top-M denote the set of top-M candidates under a frozen base scorer, and $\mathcal{W} =$ Top-M the corresponding finite candidate window used by all inference-time methods.

Model training and selection. We train a portfolio of embedding-based KGC models using PyKEEN and select a single model per dataset based on validation performance. The portfolio includes representative architectures (ComplEx, PairRE, TransE, and RotatE), trained under a consistent protocol using the sLCWA training loop with inverse triples enabled. Models are selected based on validation MRR. Table 10 summarizes the portfolio and its scale-dependent settings.

The purpose of this stage is not to optimize returned-list semantic validity, but to obtain a strong base ranking whose outputs are later controlled at inference time. Once selected, the model is *frozen*: its parameters, scores, and induced

Table 10. KGC model portfolio used for frozen-score generation. Values depend on dataset scale according to the fixed rules implemented in the training script.

Model	Emb. dim.	Epochs	Batch	Negatives	Sampler/Loss
ComplEx	256/400	200	512/1024	32	Pseudo-typed
RotatE	384/512	250	512	64	Bernoulli + NSSA
PairRE	200/300	250	768	64	Pseudo-typed
TransE	256/400	250	1024/2048	64	Bernoulli + margin

Table 11. Selected frozen scorer per dataset. Selection is based on validation MRR.

Dataset	**Model**	**Selected run**
EurostatKG	PairRE	`03_pairre_pseudotyped`
DrugBank	RotatE	`02_rotate_bernoulli_nssa`
DBpedia	ComplEx	`01_complex_pseudotyped_xlarge`

rankings remain fixed throughout all semantic-control experiments. No retraining, fine-tuning, or score recalibration is performed after selection.

Training entry points. The public code artifact provides separate training entry points for different dataset scales. EurostatKG and DrugBank are trained using a standard single-GPU script, while DBpedia uses a dedicated multi-GPU script with scale-specific settings for batching, embedding size, checkpointing, and candidate extraction. Both scripts follow the same model-selection protocol based on validation MRR; differences are purely computational and reflect dataset scale.

To ensure comparability across datasets, hyperparameters such as embedding dimension, number of epochs, and batch size are adapted to dataset size using fixed scaling rules (e.g., larger embeddings and batch sizes for larger graphs), while preserving a consistent training procedure. The exact hyperparameter values used for each dataset are recorded in the provided configuration files and validation summaries included in the supplementary materials.

Randomness and reproducibility. All stochastic components (model initialization, negative sampling, and data splits) use fixed random seeds (seed = 42 unless otherwise specified), fixing the intended stochastic protocol across runs. For fixed checkpoints and candidate windows, candidate extraction, inference-time control, and evaluation are deterministic under the stated tie-breaking; retraining may exhibit minor hardware- or CUDA-dependent numerical variation.

All reported results use the selected checkpoint shown in Table 11; no further tuning or model changes are performed after selection. Each selected model checkpoint is stored in a dedicated `best_model/` directory and reused for all downstream experiments.

Frozen-score protocol. All inference-time methods operate under a *frozen-score* regime: for each query, all methods receive identical pretrained scores and differ only in how these scores are transformed at inference time. This ensures that all comparisons isolate the effect of semantic control from model quality, training variation, and candidate generation.

Query protocol. Evaluation is performed on standard link-prediction query slots derived from test triples, including both head and tail prediction tasks. Each query is treated independently, and all methods operate on the same set of query slots.

Candidate extraction. For each query, candidates are scored using the selected frozen model. The top-M candidates under the base score form the finite candidate window $\mathcal{W} = \text{Top-}M$. All methods operate exclusively within this window, and no method can introduce candidates outside Top-M.

Candidate windows are extracted once and reused across all methods. All rankings use deterministic tie-breaking, first by score and then by a fixed secondary key (e.g., entity identifier). No additional filtering is applied during candidate extraction; all candidates in Top-M are retained and evaluated uniformly.

The candidate-window size M is fixed per dataset, as reported in Table 8. Released artifacts are versioned, and their generation is documented by the provided scripts, configurations, and fixed seeds.

Inference-time methods. All compared methods operate on the same frozen scores and candidate windows:

- **Base**: returns the top-k candidates under the frozen score.
- **HardValidation**: removes checkable violating candidates and fills from the remaining ranking.
- **AdmissibleFirst**: prioritizes admissible candidates while preserving score order within groups.
- **OptQ**: computes the minimal sufficient intervention required to satisfy quota q, if feasible.
- **OptK**: exact case of OptQ with $q = k$.
- **OptK-UA**: unknown-aware variant with graded penalties.

Reproducibility artifacts. The released materials distribute the components for this stage across the public code artifact, the linked large-artifact archive, and authorized source inputs where required:

- processed datasets and ontology sidecars (types, constraints, disjointness);
- training configurations and validation logs;
- selected model checkpoints;
- candidate-window extraction summaries;
- experiment scripts and command-line configurations.

All downstream experiments load the same frozen checkpoints and operate on identical candidate windows, ensuring a controlled and reproducible comparison. Consequently, differences between methods arise from inference-time transformations rather than training, model capacity, or candidate generation.

C.2 Frozen Scorer Performance

Table 12 reports the ranking performance of the frozen KGC scorers used to generate candidate windows. These scores are included for transparency, not as a claim of state-of-the-art KGC accuracy. The role of the scorer in our study is to provide a fixed ranking from which a candidate window is extracted on which quota controllability can be evaluated.

Table 12. Test performance of the selected frozen scorers.

Dataset	MRR	Hits@1	Hits@10
DBpedia	0.0003	0.0001	0.0006
EurostatKG	0.1680	0.1497	0.1997
DrugBank	0.5087	0.2026	0.9100

All metrics are computed under the standard filtered setting unless otherwise specified. The reported metrics are computed over the full candidate space, whereas the control framework operates on a truncated subset Top-M of these rankings. Importantly, all methods operate on identical candidate windows derived from these scores, so differences in performance reflect only inference-time control.

The variation across datasets reflects differences in scale, density, and candidate-space difficulty. DBpedia is especially challenging due to its scale (millions of entities) and the use of full-candidate evaluation without type filtering. This intentionally avoids type-filtered candidate restriction and produces an extremely difficult large-scale retrieval setting, leading to very low absolute ranking metrics. These values reflect candidate-space difficulty rather than a change in the frozen-score control protocol.

The controllability formulation operates conditionally on the retrieved candidate window Top-M and is therefore independent of absolute ranking quality, provided that admissible candidates are present within the window. Stronger scorers may increase the amount of admissible mass within Top-M and reduce the frequency of semantic violations, but they do not eliminate the need for returned-list guarantees when violations remain. Thus, base ranking accuracy affects the empirical regime in which control operates, but does not affect the correctness of the controllability formulation or its guarantees.

All candidate windows used in the experiments are extracted once from these frozen scorers and reused across all methods, ensuring identical evaluation inputs.

Role of base-model accuracy. The proposed control formulation does not rely on weak base models. Low ranking accuracy may increase the frequency of semantic violations, but it is neither required by the method nor the source of the theoretical guarantees. STC applies whenever a frozen scorer returns a ranked candidate window and an external semantic signal supplies operational candidate statuses and energy, with unknown status retained separately.

Stronger scorers may reduce the frequency of semantic violations by ranking more admissible candidates within the top-k, but they do not eliminate the need for returned-list guarantees when violations remain. In such cases, STC provides feasibility certificates and minimal-intervention diagnostics over the returned list.

Importantly, if the base ranking already satisfies the desired semantic constraints, the required intervention is zero and the method leaves the ranking unchanged. Thus, the contribution is not that weak models make errors, but that returned-list semantic validity can be certified and controlled under fixed scores, independently of model quality.

Table 13. Common experimental parameters.

Parameter	Value
Output size k	10
Quota grid q	$\{1, 3, 5, 10\}$
EurostatKG window M	20,000
DrugBank window M	5,000
DBpedia window M	5,000
Fallback policy	Base ranking, unless otherwise stated
Tie-breaking	Fixed deterministic ordering by base score, then entity ID
Retraining	None
Random seed	42

C.3 Experimental Configuration

All experiments use the same output size, quota grid, deterministic tie-breaking, and frozen-score setting. Table 13 summarizes the common parameters used across datasets and experiments. Table 14 lists the experiment suite and maps each experiment to its purpose.

Evaluation protocol. All evaluation metrics are computed under the standard filtered setting unless otherwise specified. Evaluation is performed on link-prediction query slots derived from the test split, including both head and tail prediction tasks.

Controlled comparison setting. All experiments use identical candidate windows, query sets, and evaluation protocols across methods. As a result, all comparisons isolate inference-time behavior under fixed frozen scores.

All experiments are script-driven: each experiment has a dedicated Python entry point with explicit arguments for dataset paths, candidate-window size, quotas, and evaluation settings. The public code artifact provides the scripts and configurations, while logs and large outputs are supplied through the linked large-artifact archive, enabling regeneration and verification of the reported results. All commands assume CUDA-enabled execution; CPU execution is also supported but not used in the reported experiments. No hyperparameter tuning is performed after model selection.

Example commands. The following commands illustrate representative executions of the experiment suite in Table 14. Each script implements a specific experimental setting, and all reported results are obtained by varying only the parameters shown below while keeping the underlying model and candidate windows fixed.

```
python scripts/run_exp01_blindness_diagnostics.py \
  --dataset-name DATASET \
  --processed-dir PATH_TO_PROCESSED_DATA \
  --run-dir PATH_TO_MODEL_RUN \
  --output-dir PATH_TO_OUTPUTS \
  --top-k 10 \
```

Table 14. Experiment suite and purpose.

Exp.	Purpose
EXP-01	Semantic blindness and feasibility diagnostics.
EXP-02	Exact control on blind feasible subsets.
EXP-03	Global exact return semantics and collapse.
EXP-04	Practicality and runtime diagnostics.
EXP-05	Unknown-aware global control.
EXP-06	Constraint-family ablations.
EXP-07	Quota control on blind feasible subsets.
EXP-08	Global quota-control return semantics.
EXP-09	Quota control versus filtering in the partial regime.
EXP-10	Collapse sensitivity analysis.
EXP-11	Typed entity retrieval generalization.

```
  --top-m M \
  --split test \
  --device cuda

python scripts/run_exp08_global_quota_return_semantics.py \
  --dataset-name DATASET \
  --processed-dir PATH_TO_PROCESSED_DATA \
  --run-dir PATH_TO_MODEL_RUN \
  --output-dir PATH_TO_OUTPUTS \
  --top-k 10 \
  --top-m M \
  --quotas 1 3 5 10 \
  --fallback base \
  --split test \
  --device cuda

python scripts/run_exp09_quota_vs_filtering_partial.py \
  --dataset-name DATASET \
  --processed-dir PATH_TO_PROCESSED_DATA \
  --run-dir PATH_TO_MODEL_RUN \
  --output-dir PATH_TO_OUTPUTS \
  --top-k 10 \
  --top-m M \
  --quota 5 \
  --split test \
  --device cuda
```

Hardware and environment. Experiments were run on single-GPU systems using PyTorch with CUDA acceleration. The available GPUs included an NVIDIA RTX PRO 6000 Blackwell Max-Q with 96 GB VRAM and an NVIDIA GeForce RTX 5090 Laptop GPU with 24 GB VRAM.

Reproducibility. Fixed seeds and configuration files define the intended experimental protocol. For fixed checkpoints and candidate windows, control and

Table 15. Exact control on blind feasible subsets (EXP-02).

Dataset	Viol@10	Adm@10	Pres@10	Shift@10
EurostatKG	0.000	1.000	0.190	464.40
DBpedia	0.000	1.000	0.132	200.08
DrugBank	0.000	1.000	< 0.001	934.46

Table 16. Unknown-aware control results (EXP-05).

Dataset	β	Viol@10	Adm@10	Unknown@10	Pres@10
EurostatKG	0.5	0.000	0.534	0.466	0.178
EurostatKG	1.0	0.000	0.534	0.466	0.178
DBpedia	0.5	0.000	0.286	0.714	0.710
DBpedia	1.0	0.000	0.286	0.714	0.710
DrugBank	0.5	0.000	1.000	0.000	0.024
DrugBank	1.0	0.000	1.000	0.000	0.024

evaluation are deterministic under the stated tie-breaking; model retraining may exhibit minor hardware- or CUDA-dependent numerical variation. Software versions, dependency files, commands, logs, run summaries, and raw output tables are distributed across the public code artifact and linked large-artifact archive.

D Additional Results and Reproducibility Package

D.1 Additional Results

This section reports additional experiments from the experimental suite that are not included in the main paper due to space limitations. These results complement the main evaluation by covering exact control on blind feasible subsets (EXP-02), unknown-aware global control (EXP-05), quota control on blind feasible subsets (EXP-07), and collapse sensitivity under varying candidate-window sizes (EXP-10).

Exact control on blind feasible subsets (EXP-02). Table 15 reports exact control restricted to blind feasible queries, i.e., cases where admissible candidates are present in Top-M but not selected by the base top-k. Blind feasible queries are those with $|\mathcal{A}| \geq k$ and at least one violation in the base Top-k. This setting isolates a diagnostically clean subset of within-window selection failures and evaluates the effectiveness of exact control when feasibility holds.

Exact control fully eliminates violations and recovers fully admissible outputs across all datasets, at the cost of substantial ranking perturbation, especially in perturbation-dominated regimes such as DrugBank.

Unknown-aware global control (EXP-05). Table 16 reports OptK-UA under graded unknown penalties. Results are identical across $\beta \in \{0.5, 1.0\}$ in these datasets, indicating that unknown candidates are either absent (DrugBank) or dominated by admissible or violating candidates in the candidate window.

Table 17. Quota control on blind feasible subsets (EXP-07).

Dataset	q	Viol@10	Adm@10	Pres@10	Shift@10
EurostatKG	1	0.310	0.689	0.493	493.51
EurostatKG	3	0.187	0.813	0.369	586.12
EurostatKG	5	0.141	0.859	0.323	626.49
EurostatKG	10	0.000	1.000	0.190	464.40
DBpedia	1	0.489	0.483	0.639	95.32
DBpedia	3	0.251	0.725	0.400	169.30
DBpedia	5	0.162	0.820	0.308	187.95
DBpedia	10	0.000	1.000	0.132	200.08
DrugBank	1	0.320	0.680	0.320	641.81
DrugBank	3	0.221	0.779	0.221	727.43
DrugBank	5	0.168	0.832	0.168	772.97
DrugBank	10	0.000	1.000	< 0.001	934.46

Table 18. Collapse sensitivity under varying candidate-window sizes (EXP-10).

Dataset	M	Feasible	Collapse	Pres@10
EurostatKG	5,000	0.495	1.000	0.298
EurostatKG	20,000	0.519	1.000	0.285
DBpedia	1,000	0.246	1.000	0.152
DBpedia	5,000	0.277	1.000	0.136
DBpedia	20,000	0.295	1.000	0.128
DrugBank	1,000	0.562	1.000	0.043
DrugBank	5,000	1.000	1.000	0.024

Quota control on blind feasible subsets (EXP-07). Table 17 reports quota-control behavior restricted to blind feasible queries. This setting complements the global evaluation by isolating cases where semantic correction is possible within the candidate window.

Across blind feasible subsets, increasing q consistently reduces violations and increases admissibility. The exact endpoint ($q = 10$) reaches fully admissible outputs in all datasets, while lower quotas preserve more of the original ranking.
Collapse sensitivity (EXP-10). Table 18 reports exact-endpoint collapse under the canonical binary energy and varying candidate-window sizes. Across datasets and window sizes, OptK and AdmissibleFirst coincide exactly on feasible queries, confirming that collapse is structural rather than an artifact of a specific M. Increasing M improves feasibility by expanding the admissible set within Top-M, but does not alter the collapse behavior once feasibility holds.

Global-penalty sanity checks. We evaluate a fixed global penalty baseline to contrast query-wise minimal sufficient intervention with a non-adaptive global intervention. Results are reported on blind feasible queries, matching the diagnostic setting used in EXP-02. Table 19 reports the corresponding sanity checks.

Table 19. Global-penalty sanity checks on feasible blind subsets.

Dataset	Method	Viol@10	Adm@10	Pres@10	Shift@10
EurostatKG	GlobalLinearCal	0.009	0.991	0.199	372.86
DBpedia	GlobalLinearCal	0.747	0.230	0.902	1.64
DrugBank	GlobalLinearCal	0.980	0.020	0.980	2.43

The fixed global penalty works well only when a single penalty scale happens to match the dataset, as in EurostatKG, but fails to consistently correct violations in DBpedia and DrugBank. This supports the need for query-wise minimal sufficient interventions, since a single global penalty cannot align heterogeneous query-level score–constraint interactions.

D.2 Per-Query Control Certificates

For each query and quota, STC produces a control certificate:

$$\mathcal{C} = (q, |\mathcal{A}|, \tau_q, \lambda_q^\star, \text{status}).$$

The fields are:
- q: target admissibility quota;
- $|\mathcal{A}|$: number of admissible candidates in Top-M;
- τ_q: q-th largest admissible score (defined only when feasible);
- $\lambda_q^\star$: minimum scalar intervention required to leave at most $k-q$ positive-energy candidates strictly above τ_q;
- status: Feasible if $|\mathcal{A}| \geq q$, otherwise Infeasible.

All certificate quantities are computed over the same frozen candidate window $\mathcal{W} = \text{Top-}M$ used by all methods, ensuring consistency and comparability across inference-time procedures. Given fixed scores, deterministic tie-breaking, and a fixed candidate window, the certificate is fully deterministic and reproducible at the query level.

The certificate provides a compact, per-query explanation of both feasibility and the strength of intervention required to satisfy the target. In particular, $\lambda_q^\star$ quantifies the minimum pressure required within the scalar controlled-score family. A value $\lambda_q^\star = 0$ indicates that the frozen top-k already satisfies the requested quota, so no intervention is required.

Table 20 shows real control certificates from DBpedia, illustrating both feasible queries with varying intervention strength and infeasible cases where no admissible candidates are available. This separation allows distinguishing between retrieval failure (insufficient admissible candidates in Top-M) and within-window selection failure, which is correctable via inference-time control.

Across datasets, feasible queries exhibit heterogeneous intervention strengths. Table 20 illustrates this variation in DBpedia: the two feasible examples require $\lambda_q^\star = 10.43$ and 3.22, respectively, despite sharing the same quota $q = 5$. Stricter constraint combinations or graded unknown-aware settings can change the required intervention further.

Table 20. Example control certificates from DBpedia (EXP-07, $M = 5{,}000$).

query_id	q	$\lvert\mathcal{A}\rvert$	τ_q	$\lambda_q^\star$	**Status**
`test_head_000000006`	5	5	14.83	10.43	FEASIBLE
`test_tail_000000026`	5	875	31.51	3.22	FEASIBLE
`test_head_000000010`	5	0	$\perp$	$\perp$	INFEASIBLE

In contrast, infeasible queries are explicitly identified by $|\mathcal{A}| < q$, preventing ill-posed control objectives and enabling principled fallback strategies when the target is infeasible. The certificate therefore serves both as a diagnostic tool and as an interpretable interface between ranking behavior and semantic constraints.

D.3 Reproducibility Checklist

Together, the public code artifact and linked large-artifact archive provide the redistributable components needed to inspect and regenerate the reported experiments; restricted source inputs require authorized access:

- source code for all experiment scripts;
- configuration files and representative command-line arguments;
- dependency/environment files specifying software versions;
- dataset split metadata and preprocessing summaries;
- ontology-derived constraint files (types, domain/range, disjointness);
- frozen model identifiers and validation-selection summaries;
- candidate-window extraction summaries;
- raw summary tables for all reported experiments;
- per-query control certificates for feasibility and intervention analysis;
- scripts used to generate all tables reported in the paper.

Reproducibility scope. The released materials support direct inspection of the publication-level results and end-to-end regeneration where the required source inputs and compute are available. Given the same inputs, configurations, and compatible software environment, the pipeline regenerates the candidate windows, control outputs, and evaluation metrics. Deterministic tie-breaking and fixed random seeds are used throughout; minor hardware- or CUDA-dependent numerical variation may remain.

Large artifacts. The separately hosted `large-artifact archive` contains the redistributable full processed datasets, frozen model checkpoints, materialized candidate-window files, full query-level outputs, verified experiment outputs, and execution records used in the paper. The archive is versioned and accompanied by file manifests and checksums.

Restricted resources. Restricted source resources, including the DrugBank XML dump, are not redistributed. Users with authorized access can regenerate the corresponding derived artifacts using the supplied extraction scripts and configurations, subject to the original licenses.

Determinism and variability. All experiments use deterministic tie-breaking and fixed seeds. Minor numerical variation may occur due to hardware and CUDA execution, but does not affect the reported conclusions.

D.4 Codebase and Artifact Structure

The supporting materials are organized around the stages required to inspect and reproduce the experimental pipeline: preprocessing, model training, candidate extraction, and inference-time control. The public GitHub repository contains code, scripts, configurations, lightweight metadata, curated result tables, and provenance checks. The separately hosted large-artifact archive contains the redistributable large inputs and outputs used in the paper.

Supporting artifacts. The lightweight code artifact is available through the public GitHub repository, while the redistributable large inputs and outputs are available through the separately hosted Drive archive.

```
          GitHub code artifact   |  large-artifact archive

semantic-target-control/
|-- data/
|   |-- raw/
|   |   |-- dbpedia/
|   |   |-- drugbank/
|   |   `-- eurostatkg/
|   `-- processed/
|       |-- dbpedia/
|       |-- drugbank/
|       `-- eurostatkg/
|-- runs/
|   |-- dbpedia_frozen_model/
|   |-- drugbank_frozen_model/
|   `-- eurostatkg_frozen_model/
|-- outputs/
|   |-- dbpedia_results/
|   |-- drugbank_results/
|   `-- eurostatkg_results/
|-- scripts/
|   |-- build_dbpedia_eval_subset.py
|   |-- preprocess_dbpedia.py
|   |-- preprocess_drugbank_xml.py
|   |-- preprocess_eurostatkg.py
|   |
|   |-- train_base_kgc.py
|   |-- train_schema_aware_portfolio_multigpu.py
|   |
|   |-- run_all_drugbank.ps1
|   |-- run_all_eurostat.ps1
|   |-- run_dbpedia_subset_suite.ps1
|   |
|   |-- run_exp01_blindness_diagnostics.py
```

```
|   |-- run_exp02_exact_blind_subset_bench.py
|   |-- run_exp03_global_return_semantics_exact.py
|   |-- run_exp04_practicality_exact.py
|   |-- run_exp05_unknown_aware_global.py
|   |-- run_exp06_ablations_exact.py
|   |-- run_exp07_quota_blind_subset_bench.py
|   |-- run_exp08_global_quota_return_semantics.py
|   |-- run_exp09_quota_vs_filtering_partial.py
|   |-- run_exp10_collapse_sensitivity_analysis.py
|   `-- run_exp11_typed_entity_retrieval_generalization.py
`-- commands/
```

Directory roles.

- `data/raw/`: Raw or source-level dataset inputs where redistribution is permitted; restricted sources such as the DrugBank XML dump are not included.
- `data/processed/`: Processed splits, mappings, entity types, and ontology-derived constraint files used by all downstream components.
- `runs/`: Frozen-model identifiers, validation-selection summaries, and pointers to large checkpoint artifacts.
- `outputs/`: Summary tables, candidate-window summaries, feasibility diagnostics, and per-query control certificates.
- `scripts/`: End-to-end pipeline components, including preprocessing, model training, candidate extraction, and experiment execution.
- `commands/`: Representative command lines used to reproduce the reported experiments.

Pipeline structure. The repository follows a linear experimental pipeline:

$$\texttt{raw data} \to \texttt{processed artifacts} \to \texttt{trained model} \to \texttt{candidate windows} \to \texttt{control outputs}.$$

Each stage is implemented by dedicated scripts in `scripts/`, and intermediate artifacts are stored explicitly, enabling step-by-step reproducibility and inspection.

Artifact access. The public GitHub repository contains the lightweight reproducibility materials: source code, preprocessing, training, and experiment scripts, configuration files, environment specifications, preprocessing metadata, split summaries, ontology-derived constraint files, curated publication-level tables, provenance checks, and representative diagnostic outputs. The separately hosted `large-artifact archive` contains the redistributable large artifacts, including full processed datasets, frozen model checkpoints, complete candidate-window files, full query-level outputs, verified experiment outputs, and execution records. Restricted raw resources, such as the DrugBank XML dump, are not redistributed; users with authorized access can reconstruct the corresponding derived dataset using the supplied scripts and configurations.

Reproducibility entry points. The main experiments are reproduced through the dedicated scripts in `scripts/`. Representative commands are provided in Section C.3. For example, the global quota-control experiment is run with:

```
python scripts/run_exp08_global_quota_return_semantics.py \
  --dataset-name DATASET \
  --processed-dir PATH_TO_PROCESSED_DATA \
  --run-dir PATH_TO_MODEL_RUN \
  --output-dir PATH_TO_OUTPUTS \
  --top-k 10 \
  --top-m M \
  --quotas 1 3 5 10 \
  --fallback base \
  --split test \
  --device cuda
```

E Extended Empirical Evidence

The main paper reports the canonical reviewer-critical comparisons. This annex organizes the additional evidence retained from the broader experimental campaign. The analyses address four complementary questions: whether the canonical population and policy scope are comparable, how quota strictness and candidate-window size affect the control regime, whether results are concentrated in a small number of relations, and whether head and tail prediction behave differently. All tables use the same frozen scores, materialized candidate windows, deterministic tie-breaking, and operational semantic-status protocol as the main evaluation.

E.1 Evaluation Scope and Compared Policies

Table 21 records the canonical $q = 5$ evaluation population and exposes the main retrieval-side limitation: DBpedia has extremely low factual-target exposure despite a large candidate window. This quantity is reported separately from semantic feasibility because the controller can repair within-window selection failures but cannot recover a factual target absent from the materialized window.

Table 21. Canonical $q = 5$ evaluation settings. DBpedia uses the fixed paper subset. Feas. denotes finite-window feasibility and Target@M the factual-target exposure rate.

Dataset	Scorer	Entities	Queries	M	Feas.	Target@M
EurostatKG	PairRE	44,948	12,922	20,000	0.780	0.543
DBpedia	ComplEx	5,496,987	47,246	5,000	0.687	0.0014
DrugBank	RotatE	22,466	585,952	5,000	1.000	0.571

Table 22 clarifies the role of each returned-list policy. Hard validation is a legitimate strict endpoint, QuotaFill is a direct quota repair baseline, fixed and learned policies provide non-certified operating points, and OptQ is the only policy in this comparison that returns an explicit query-level feasibility and intervention certificate.

Table 22. Compared systems and returned-list policies. All post-processing policies operate on identical frozen scores and candidate windows.

Policy	Mechanism	Query-specific	Certificate and role
Base KGE	Frozen PairRE, ComplEx, or RotatE ranking	No	Upstream ranker; no control certificate
HardVal	Remove candidates that fail operational validation	No	Strict validation endpoint
QuotaFill	Replace visible items lacking admissible status until quota q is reached	Yes	Direct quota-repair baseline
Fixed-λ	Apply one global semantic penalty to every query	No	No query-level guarantee
Learned reranker	Apply a learned semantic returned-list policy	No	Empirical operating point
OptQ	Compute the minimum pressure for the requested quota	Yes	Feasibility and quota certificate

E.2 Learned-Policy Protocol and Additional Scorers

The learned baseline is a candidate-level LightGBM classifier. Its features are `base_score`, score normalization and gap features, reciprocal and log-rank features, rank percentile and base-top-k membership, operational admissible/violating/unknown/checkable indicators, semantic energy, relation identifier, and head/tail prediction mode. Candidate labels follow the admissibility policy, and class-balanced sample weights are used during fitting. The canonical implementation uses 300 trees, learning rate 0.05, 31 leaves, unbounded depth, row and column subsampling of 0.9, and seed 42.

Training, validation, and evaluation queries are built from disjoint named splits. EurostatKG uses 2,000 training and 800 validation queries; DBpedia and DrugBank use 3,000 and 800, respectively. The held-out evaluation samples contain 1,000 EurostatKG, 1,000 DBpedia, and 10,000 DrugBank queries before selection-failure filtering. The frozen mixing-weight grid is $\{0, 0.03, 0.1, 0.3, 1, 3, 10, 30\}$. The complete validation summaries, feature importances, and held-out summaries are included in the artifact. The main paper reports two diagnostic grid operating points—preservation-oriented and quota-reliable—rather than claiming an independently optimized learning-to-rank system.

The learned and fixed-pressure experiments intentionally have different computational scopes. Their tables are internally matched and include separate OptQ reference rows; absolute OptQ values across those tables are not treated as replicates. Table 23 preserves the complete learned-policy diagnostics, including the rank-shift column omitted from the compact main-paper comparison.

Table 23. Full learned-policy diagnostics on the held-out sampled matched selection-failure populations used in the main paper. Q denotes quota success; Shift@10 preserves the full diagnostic column.

Dataset	Policy	Q	Viol@10	Adm@10	Pres@10	Shift@10
EurostatKG	Learned-nearPres	0.711	0.384	0.616	0.456	114.656
	OptQ	1.000	0.497	0.503	0.569	173.617
	Learned-safe	1.000	0.000	1.000	0.072	621.519
DBpedia	Learned-nearPres	0.725	0.383	0.610	0.531	17.059
	OptQ	1.000	0.518	0.468	0.673	86.976
	Learned-safe	1.000	0.067	0.931	0.210	289.585
DrugBank	Learned-nearPres	0.212	0.772	0.228	0.773	95.952
	OptQ	1.000	0.500	0.500	0.500	397.295
	Learned-safe	1.000	0.000	1.000	0.000	851.811

Table 24 reports the semantic diagnostics for the three additional full-scope scorer runs, while Table 25 reports their factual-target exposure and factual-utility deltas. They cover EurostatKG with ComplEx and DrugBank with ComplEx and PairRE; we make no additional-scorer claim for DBpedia.

Table 24. Additional full-scope $q = 5$ frozen-scorer runs. The semantic effect retains the same direction beyond the three canonical scorer–dataset pairs.

Dataset	Scorer	Feas.	Target@M	ΔViol@10	ΔAdm@10	Pres@10
EurostatKG	ComplEx	0.780	0.427	-0.276	+0.276	0.724
DrugBank	ComplEx	1.000	0.168	-0.179	+0.179	0.821
DrugBank	PairRE	1.000	0.618	-0.124	+0.124	0.876

Table 25. Factual-target exposure and factual-utility deltas for the three additional full-scope scorer runs. These values complement the semantic cross-scorer diagnostics in Table 24.

Dataset	Scorer	Target@M	ΔHit@10	ΔMRR@10
EurostatKG	ComplEx	0.427411	+0.002476	+0.000930
DrugBank	ComplEx	0.168475	+0.000039	+0.000012
DrugBank	PairRE	0.617634	+0.000415	+0.000074

E.3 Quota and Candidate-Window Sensitivity

The quota frontier in Table 26 separates control strength from preservation. Increasing q raises admissibility but requires a weakly larger intervention and generally reduces overlap with the frozen top-k. The $q = 10$ row is the strict endpoint; lower quotas represent the intended partial-control regime. Table 27 complements this analysis by varying the upstream candidate-window size while

keeping the deployment rule fixed. Larger windows improve finite-window feasibility by exposing more admissible candidates, whereas smaller windows leave more queries on the frozen fallback. Once the window is materialized, however, the controller does not re-query or expand it. The two wide tables are placed on consecutive full-page rotated floats with identical orientation and captions above the tables.

Table 26. OptQ quota frontier. Feas. is measured over the canonical query population; the remaining metrics are computed on feasible queries, for which OptQ satisfies the requested quota. The $q = 10$ row is the strict quota endpoint.

Dataset	q	Feas.	Adm@10	Pres@10	Shift@10	Median $\lambda^\star$
EurostatKG	1	0.780	0.260	0.950	14.8	0.001
EurostatKG	3	0.780	0.391	0.819	71.3	0.023
EurostatKG	5	0.780	0.554	0.656	166.3	0.045
EurostatKG	10	0.780	1.000	0.210	556.4	0.252
DBpedia	1	0.729	0.391	0.966	14.6	0.000
DBpedia	3	0.704	0.501	0.868	46.2	0.414
DBpedia	5	0.687	0.636	0.742	72.0	1.587
DBpedia	10	0.673	1.000	0.386	184.4	5.353
DrugBank	1	1.000	0.122	0.902	76.0	2.195
DrugBank	3	1.000	0.317	0.707	244.6	2.350
DrugBank	5	1.000	0.512	0.512	425.6	2.501
DrugBank	10	1.000	1.000	0.024	912.1	3.010

Table 27. Candidate-window sensitivity at $q = 5$ under the all-query deployment protocol. Feas. is measured over the canonical population; the frozen top-10 is retained when the quota is infeasible.

Dataset	M	Feas.	ΔViol@10	ΔAdm@10	Pres@10	Shift@10
EurostatKG	500	0.522	-0.140	+0.140	0.860	10.3
EurostatKG	1,000	0.626	-0.191	+0.191	0.809	32.6
EurostatKG	5,000	0.779	-0.268	+0.268	0.732	126.8
EurostatKG	10,000	0.780	-0.269	+0.269	0.731	129.8
EurostatKG	20,000	0.780	-0.269	+0.269	0.731	129.8
DBpedia	500	0.592	-0.133	+0.130	0.870	7.4
DBpedia	1,000	0.637	-0.156	+0.152	0.848	17.3
DBpedia	2,500	0.673	-0.175	+0.170	0.830	33.5
DBpedia	5,000	0.687	-0.183	+0.177	0.823	49.4
DrugBank	500	0.195	-0.085	+0.085	0.915	24.0
DrugBank	1,000	0.644	-0.310	+0.310	0.690	179.4
DrugBank	2,500	0.982	-0.479	+0.479	0.521	401.0
DrugBank	5,000	1.000	-0.488	+0.488	0.512	425.6

Across datasets, the quota frontier confirms that partial targets avoid the over-control induced by forcing every positive-energy candidate below the q-th admissible threshold. The dataset-specific intervention scales also show why a single global penalty cannot provide a uniform certified policy. Candidate-window sensitivity is complementary: expansion addresses retrieval failure upstream, while OptQ addresses within-window selection failure and preservation downstream.

E.4 Structural Overview

Table 28 summarizes relation support, concentration, and weighted returned-list utility. Table 29 provides a compact head–tail view on the matched selection-failure subset. These summaries motivate the detailed decompositions that follow.

Table 28. Relation-level structural diagnostics at $q = 5$. Coverage and concentration use support-filtered feasible queries.

Dataset	Scorer	Rel. cov.	Max	Top-3	Wt. Adm.	Wt. Pres.
EurostatKG	PairRE	0.911	0.194	0.495	0.551	0.660
DBpedia	ComplEx	0.936	0.161	0.327	0.639	0.745
DrugBank	RotatE	1.000	0.975	0.996	0.512	0.512

Table 29. Head–tail structural summary on the matched selection-failure subset. Each metric pair reports admissibility followed by preservation.

Dataset	Scorer	Head Adm. / Pres.	Tail Adm. / Pres.
EurostatKG	PairRE	0.509 / 0.615	0.504 / 0.629
DBpedia	ComplEx	0.508 / 0.672	0.505 / 0.634
DrugBank	RotatE	0.500 / 0.500	0.500 / 0.500

DrugBank is strongly concentrated in a small number of relation signatures, whereas EurostatKG and DBpedia distribute feasible support more broadly. Consequently, both macro and support-weighted views are needed before drawing dataset-level conclusions.

E.5 Relation-Level Diagnostics

Table 30 reports the retained support and concentration explicitly, and Table 31 separates macro admissibility from support-weighted admissibility and preservation. The paired presentation prevents high-support relations from silently dominating the interpretation.

Table 30. Relation-support and concentration diagnostics at $q = 5$. Coverage is the retained feasible-query support.

Dataset	Scorer	Relations	Coverage	Max share	Top-3 share
EurostatKG	PairRE	8	0.911	0.194	0.495
DBpedia	ComplEx	113	0.936	0.161	0.327
DrugBank	RotatE	6	1.000	0.975	0.996

Table 31. Relation-level returned-list utility on the same support-filtered feasible queries.

Dataset	Macro Adm@10	Weighted Adm@10	Weighted Pres@10
EurostatKG	0.569	0.551	0.660
DBpedia	0.610	0.639	0.745
DrugBank	0.872	0.512	0.512

The macro–weighted differences are most pronounced for DrugBank, consistent with its highly concentrated relation distribution. EurostatKG and DBpedia show broader support and smaller concentration, so their aggregate outcomes are less dependent on a single dominant relation.

E.6 Head–Tail Diagnostics

Table 32 reports all feasible queries, while Table 33 restricts the comparison to matched within-window selection failures. Reporting both views distinguishes population composition from direction-specific controller behavior.

Table 32. Head–tail slices on feasible queries at $q = 5$. The scorer is PairRE for EurostatKG, ComplEx for DBpedia, and RotatE for DrugBank.

Dataset	Mode	Queries	Viol@10	Adm@10	Pres@10	Shift@10
EurostatKG	Head	6,434	0.446	0.553	0.649	208.7
EurostatKG	Tail	3,651	0.444	0.555	0.668	91.4
DBpedia	Head	16,159	0.293	0.698	0.799	35.7
DBpedia	Tail	16,277	0.413	0.575	0.686	107.9
DrugBank	Head	292,926	0.487	0.513	0.513	551.2
DrugBank	Tail	292,976	0.489	0.511	0.511	300.1

Table 33. Head–tail slices on the matched selection-failure subset at $q = 5$.

Dataset	Mode	Queries	Viol@10	Adm@10	Pres@10	Shift@10
EurostatKG	Head	5,855	0.490	0.509	0.615	229.4
EurostatKG	Tail	3,275	0.495	0.504	0.629	101.9
DBpedia	Head	9,917	0.478	0.508	0.672	58.2
DBpedia	Tail	13,970	0.482	0.505	0.634	125.7
DrugBank	Head	285,544	0.500	0.500	0.500	565.4
DrugBank	Tail	286,328	0.500	0.500	0.500	307.1

EurostatKG is comparatively balanced across prediction directions. DBpedia shows stronger head-side admissibility and preservation on the full feasible population, while the matched subset narrows the admissibility difference but retains a preservation gap. DrugBank remains nearly symmetric in admissibility, with larger head-side displacement.

E.7 Summary of the Extended Evidence

Taken together, the extended results support three bounded conclusions. First, stricter quotas improve returned-list admissibility at a measurable preservation cost rather than constituting a free accuracy gain. Second, candidate-window size affects finite-window feasibility but does not change the controller's fixed-window deployment assumptions. Third, the relation and head–tail analyses show that aggregate effects are not interpreted without support and direction diagnostics.

F Semantic-Evidence Coverage Inventory

The operational labels used by STC depend on the semantic evidence available after preprocessing. Table 34 reports all 26 preflight fields selected by the type, schema, constraint, and unknown-coverage extractor. The inventory is deliberately lossless: exact field names and values are retained so that missing evidence is not silently reinterpreted as a confirmed violation.

Table 34: Exact preflight semantic-evidence coverage inventory.

Dataset	Field	Exact value
DBpedia	`counts.avg_types_per_typed_entity`	`3.8654481783820525`
DBpedia	`counts.typed_entities`	`4628358`
DBpedia	`counts.typing_coverage`	`0.8419808888032663`
DBpedia	`schema.classes_in_closure`	`767`
DBpedia	`schema.classes_with_subclass_info`	`760`

Continued on next page

Table 34 (continued)

Dataset	Field	Exact value
DBpedia	schema.relations_with_domain_assertions	2421
DBpedia	schema.relations_with_range_assertions	2588
DBpedia	schema.subclass_edges	769
DBpedia	source_files.instance_types	"data\raw\DBpedia\instance_types_en.ttl.bz2"
DrugBank	counts.typed_entities	22466
DrugBank	counts.typing_coverage	1.0
DrugBank	schema.disjoint_pairs	"<listlength=3>"
DrugBank	schema.relation_signatures.drug_has_carrier	["Drug","Protein"]
DrugBank	schema.relation_signatures.drug_has_enzyme	["Drug","Protein"]
DrugBank	schema.relation_signatures.drug_has_target	["Drug","Protein"]
DrugBank	schema.relation_signatures.drug_has_transporter	["Drug","Protein"]
DrugBank	schema.relation_signatures.drug_in_pathway	["Drug","Pathway"]
DrugBank	schema.relation_signatures.drug_interacts_with	["Drug","Drug"]
DrugBank	schema.schema_source	"induced_from_xml_structure"
EurostatKG	counts.typed_entities	44704
EurostatKG	counts.typing_coverage	0.994571504850049
EurostatKG	schema_triples.owl_disjointWith	0
EurostatKG	schema_triples.rdf_type	67895
EurostatKG	schema_triples.rdfs_domain	32
EurostatKG	schema_triples.rdfs_range	36
EurostatKG	schema_triples.rdfs_subClassOf	1924

DBpedia has substantial but incomplete typing coverage and a large ontology-derived closure; DrugBank has complete induced typing under its six XML-derived relation signatures; and EurostatKG has near-complete typing but no retained `owl:disjointWith` triples in the processed bundle. These differences explain why unknown status is materially more prominent in DBpedia than in the other two datasets.

References

1. Lehmann, J., Isele, R., Jakob, M., Jentzsch, A., Kontokostas, D., Mendes, P.N., Hellmann, S., Morsey, M., van Kleef, P., Auer, S., Bizer, C.: DBpedia–A Large-scale, Multilingual Knowledge Base Extracted from Wikipedia. Semantic Web **6**(2), 167–195 (2015). https://doi.org/10.3233/SW-140134
2. Vassiliades, A., Bassiliades, N., Meditskos, G., Spiliopoulos, K.: Building eurostat knowledge graph. In: Proceedings of the 14th International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management, Volume 2: KEOD. pp. 128–135. SciTePress (2022). https://doi.org/10.5220/0011527100003335
3. Wishart, D.S., Feunang, Y.D., Guo, A.C., Lo, E.J., Marcu, A., Grant, J.R., Sajed, T., Johnson, D., Li, C., Sayeeda, Z., Assempour, N., Iynkkaran, I., Liu, Y., Maciejewski, A., Gale, N., Wilson, A., Chin, L., Cummings, R., Le, D., Pon, A., Knox, C., Wilson, M.: DrugBank 5.0: A major update to the DrugBank database for 2018. Nucleic Acids Research **46**(D1), D1074–D1082 (2018). https://doi.org/10.1093/nar/gkx1037